

Software Project Management – IS212

Automated Testing #1 & Continuous Integration (CI)

Announcements

Announcements – Week 7 Consultations

- In Week 7, each group will have a **20 min (ungraded) consult** with us
 - *Details to be sent soon by email / Slack*
- We will discuss your **software development process** and ask you about your **first sprint**, e.g.:
 - *What does your product backlog look like?*
 - *How did your first sprint go? How are you improving upon it?*
 - *What tool(s) are you using to manage your process?*
 - *Are you on track to deliver the first release?*
- We will **give feedback on your process** and are happy to answer questions about how to improve it
 - *Note that this is NOT a customer clarification session*



Announcements – Week 9 Quiz

- **Week 5 Quiz results are out** – *please ask us if any doubts*
 - *Of the 4 quizzes, only your top 3 quizzes retained*
- Our third quiz will take place at the **start of your Week 9 class**
 - *Starts promptly at the beginning of class!*
 - *No extra time if you arrive to class late*
- Need to be physically present (no make-ups; see Week 1 slides)
- Requires Respondus LockDown Browser
- Short (~10 mins); covers the **Weeks 5–6 official materials**
 - *MCQ-style questions (no written)*

Announcements – Forthcoming Assignments

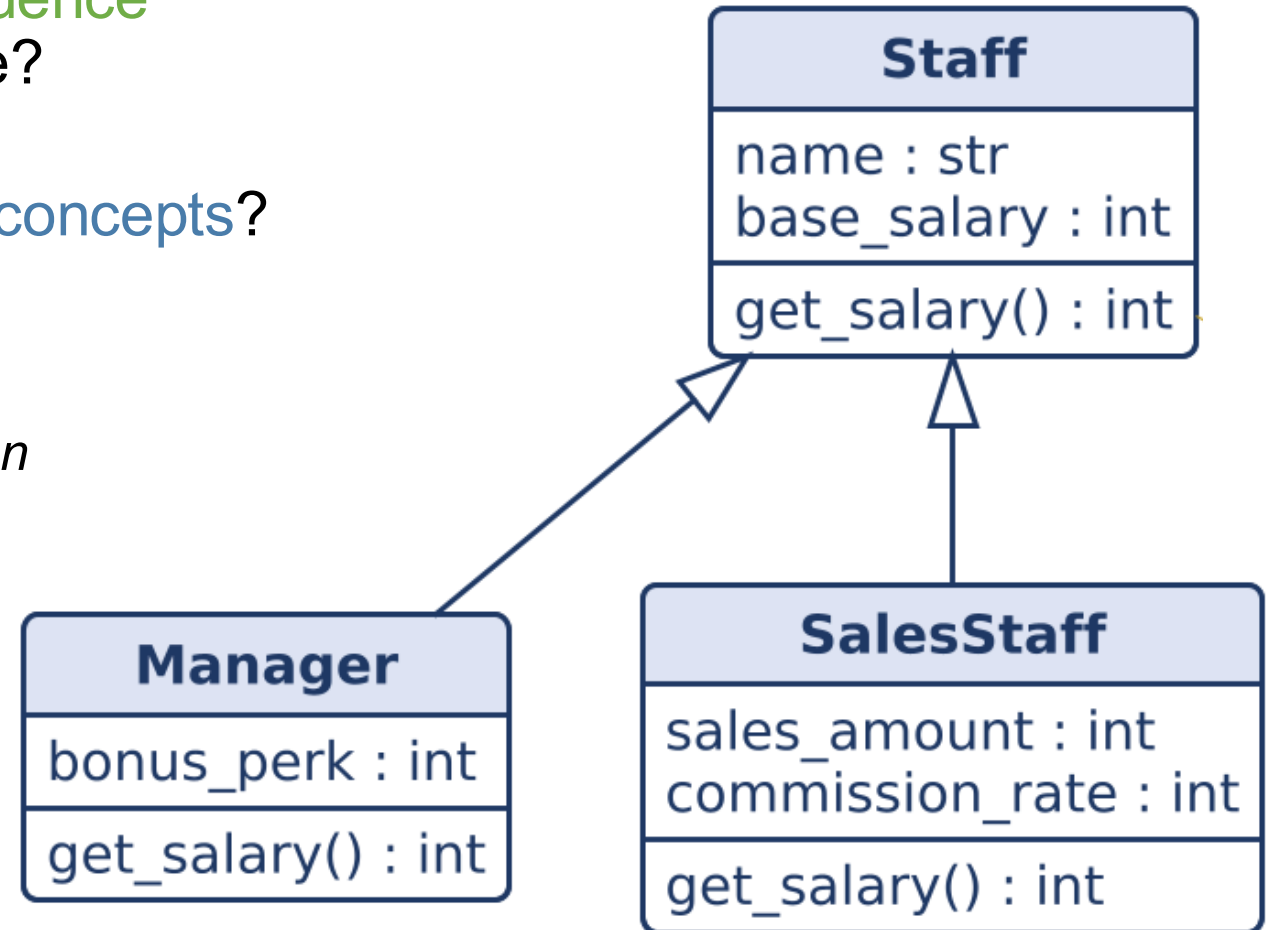
- **Individual – Automated Testing Assignment (2.5%)**
 - *Unit testing exercise for some given code; **due Friday Week 7 (23:59)***
 - *(More details at the end of today's class)*
- **Group – Topic Research & Sharing (3%)**
 - **Week 10:** 7-min presentation + 2-min Q&A
 - *Research a specific, current topic beyond class material*
 - *Include a demo / real output + critical assessment*
 - Reserve your topic **by Friday Week 7 (23:59)**



Recap

Week 5 Recap – Class Diagrams

- What do **UML class diagrams** and **sequence diagrams** allow us to model about code?
- What are the following **object-oriented concepts**?
 - *Data abstraction*
 - *Encapsulation*
 - *Association, Composition, and Aggregation*
 - *Inheritance*
 - *Polymorphism*



Week 5 Recap – Collaborating with Git

- What is a **branch**, and why do we prefer **short-lived branches**?
- What is a **pull request**, and what can it be linked to?
- Who – or what – can **review** a pull request?
- How does **trunk-based development** differ from **feature-based development**?
- What causes a **merge conflict**, and how do we resolve one?

*How do we ensure a commit
doesn't “break” the system?*

Today: Automated Testing (Part 1) & CI

Objectives

On completing this class, you will be able to:

- Implement **unit tests** that satisfy FIRST
- Conduct **Test-Driven Development (TDD)** and explain its benefits
- Use a **Python testing framework** to write well-tested, clean code
- Judge whether human- or AI-generated tests are **convincing evidence**

Outline

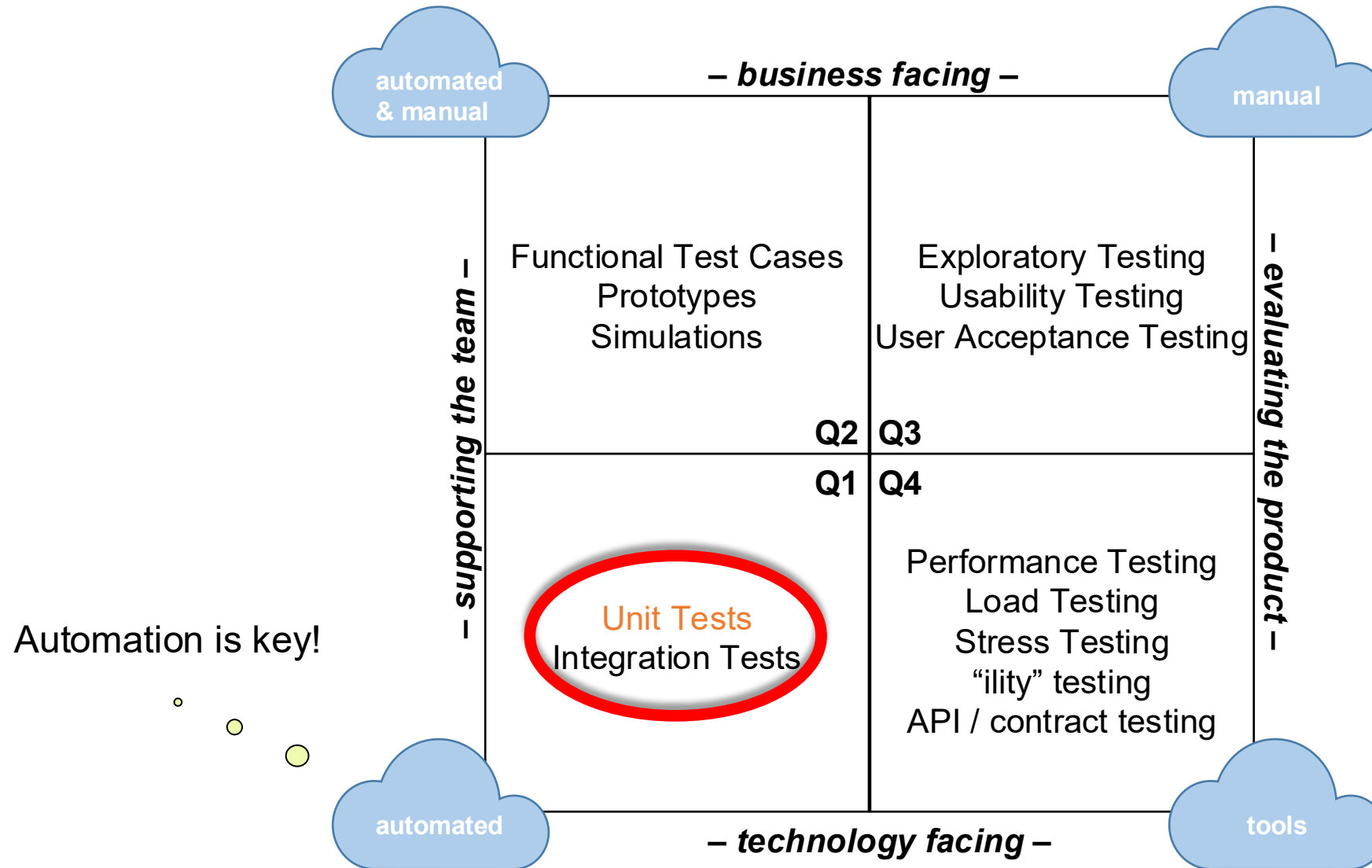
- **Test Automation** – testing pyramid, unit tests, FIRST, coverage
- **Evidence** – what generated tests prove, and how to review them
- **TDD** – Red-Green-Refactor, TDD in scrum, worked examples
- **unittest** – assertions, Arrange-Act-Assert, test structure
- **CI** – pipelines, runners, status checks



GitHub Actions

Test Automation

Recap – Agile Testing Quadrant



Why Automate Tests?

Repeatable and One-Click

Early Feedback



Safety for Refactoring

Guardrails for Coding Agents

Automated Testing – Example

```
def is_palindrome(s):  
    cleaned_string = ''.join(char.lower() for char in s if char.isalnum())  
    return cleaned_string == cleaned_string[::-1]
```



What might automated tests look like for this function ... ?

```
assert is_palindrome("radar")
```

```
assert not is_palindrome("Phris")
```

```
assert is_palindrome("")
```

```
assert not is_palindrome("123")
```

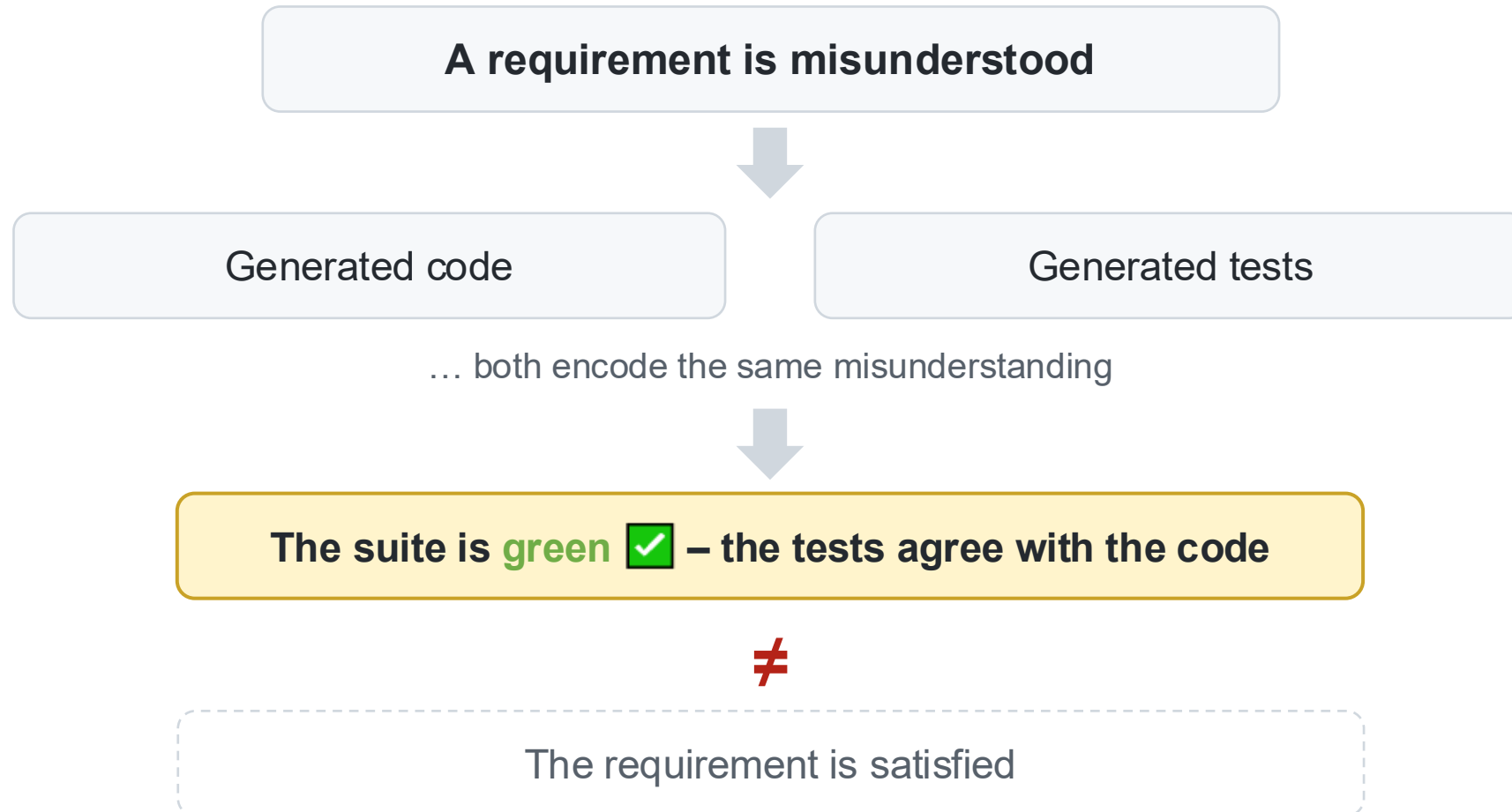
```
assert is_palindrome("A Santa lived as a devil at NASA")
```

NB: the assertions are **test oracles**, i.e. mechanisms used to determine whether the program's output is correct

AI Changes the Bottleneck

- Previously the **constraint** was **producing** software
 - *Writing code was expensive; writing tests was laborious*
 - *A **green test suite**  was evidence that somebody **had done careful work***
- Increasingly, the constraint is **confirming that the software is right**
 - *Generating candidate code is cheap. Generating plausible tests is cheap*
 - *A **green suite**  may only mean the tests were **written to fit the code***
- The bottleneck has moved to **specification, review, and validation**

When the Same Author Writes the Code and the Tests



A test suite cannot catch a **mistake it shares** with the code

When the Same Author Writes the Code and the Tests

Example: “Apply a 10% discount to orders over \$100”

AI-Generated Code:

```
def final_price(total):  
    return total * 0.90 if total >= 100 else total
```

AI-Generated Test:

```
def test_final_price():  
    assert final_price(100) == 90
```

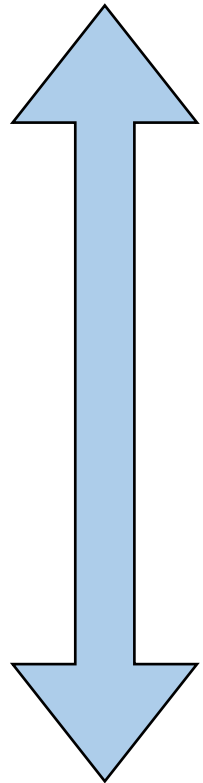
The **test passes**  because it encodes the **same misunderstanding** as the code: the requirement says **over \$100**, but both implement **\$100 or more**

Tests Are Executable Evidence

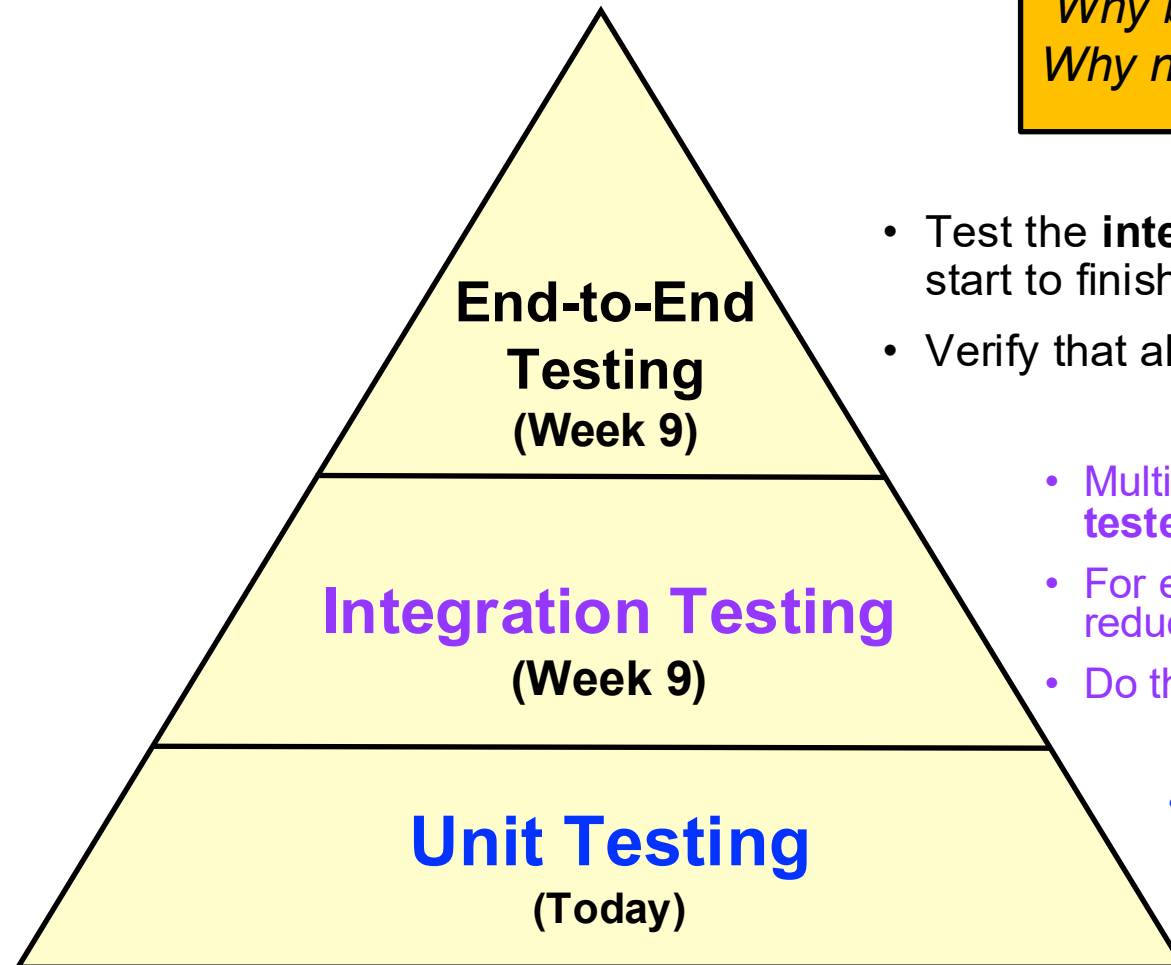
- A **test suite** is an argument that the **implementation does what was asked**
- Ask of every test: **what would have to be broken for this to fail?**
 - *If the answer is “nothing plausible”, the test is **decoration***
- Coding agents make it easier to **produce** code and tests
- They do not make it easier to **know that the code is correct**
- **So:** what makes test evidence trustworthy? That is the rest of today.

Automated Testing Pyramid

- more “realistic” tests
- slower, more brittle



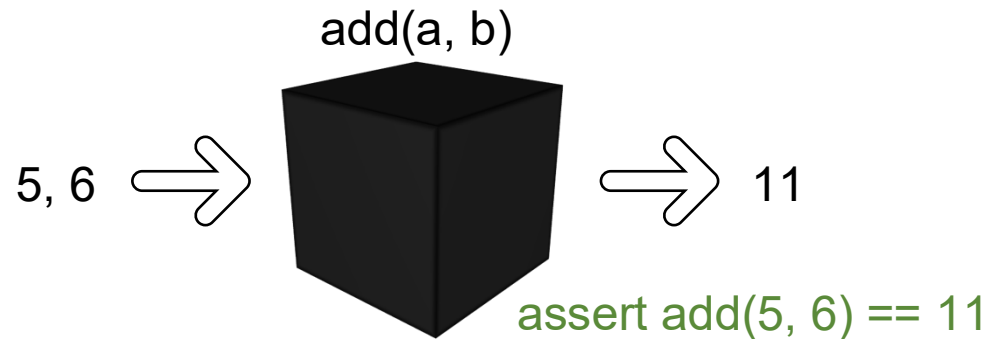
- faster, more reliable
- smaller scope



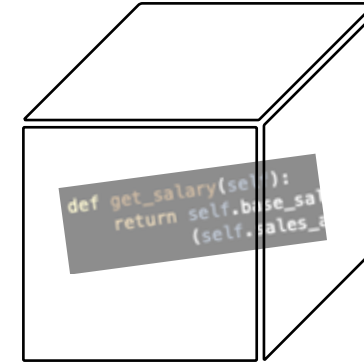
*Why bother with unit tests?
Why not just do end-to-end?*

- Test the **integrated flow** of the application from start to finish, simulating real user scenarios
- Verify that all subsystems work together correctly
- Multiple components (modules) of a system are **tested together**
- For example, “does placing an **order** correctly reduce the remaining **inventory**?”
- Do the components correctly integrate with the DB?
- Individual “units” (functions/classes) are tested **in isolation** from external systems and significant collaborators (databases, networks, ...)

Designing Tests – Black Box vs. White Box

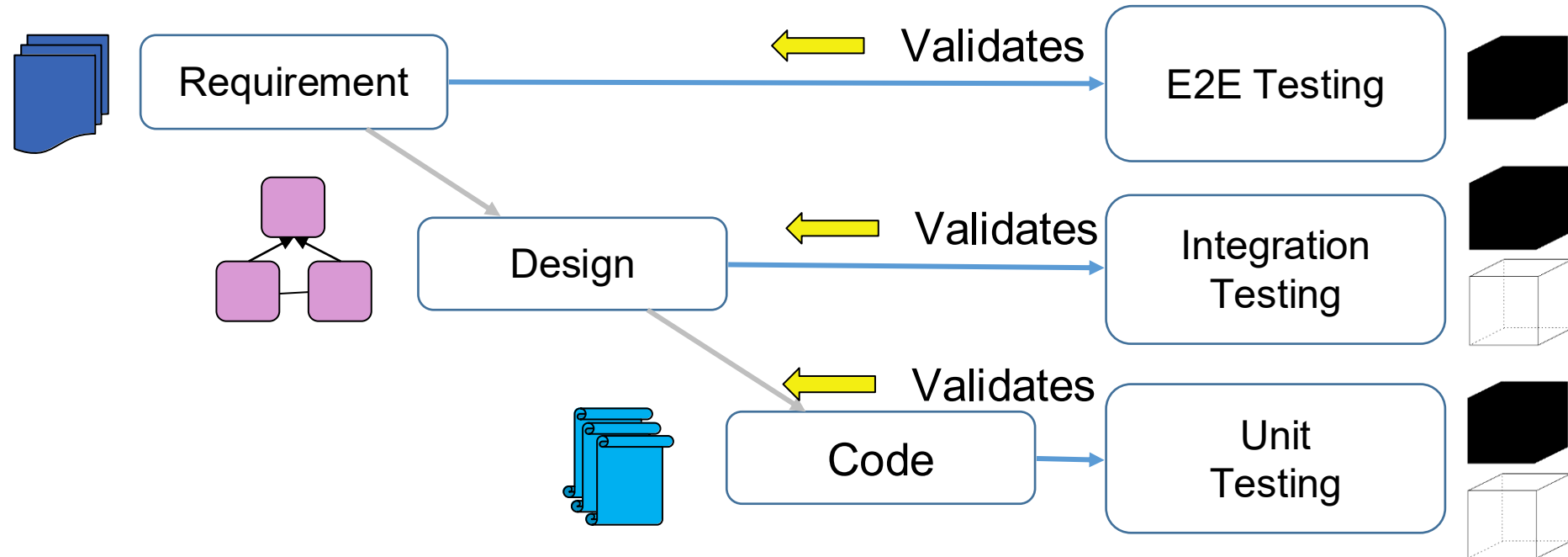


- Design tests **based on inputs / outputs** without considering the implementation
- Focus on **observed behaviour** rather than the internal workings of the code



- Take the **implementation of functions** (i.e. source code) into consideration when designing tests
- Ensure that all **internal paths, branches, and conditional statements** are working correctly

What Tests Validate



Which levels are black/white box testing suitable for?

FIRST Principles of Testing

- The “**FIRST**” mnemonic summarises the **expected qualities** of automated tests

F *ast*

I *solated*

R *epearable*

S *elf-Validating*

T *imely*

FIRST Principles of Testing

- **Fast** – production systems have 1000s of tests; need to be **fast**
- **Isolated** – should be possible to **execute tests in any order** (a test should not fail because a certain other test was run first)
- **Repeatable** – executing the **same test multiple times** shouldn't change the outcome, otherwise it is a **flaky** test
- **Self-Validating** – test cases themselves should **determine their outcome** by using assertions (no manual inspection required)
- **Timely** – **tests should be written just before or alongside the production code** – not bolted on as an afterthought

Frameworks for Automated Tests

- Rather than writing **ad hoc assert** statement, we typically make use of **testing frameworks** that provide useful features:
 - *Test runners, automated test discovery, test fixtures, advanced assertions, ...*
- For **unit and integration tests** in Python, we will use “**unittest**”, but another popular choice is “**pytest**”
 - *For other languages, see: JUnit, NUnit, MSTest, CppUnit, PHPUnit, etc.*



Test-Driven Development

Test-Driven Development (TDD)

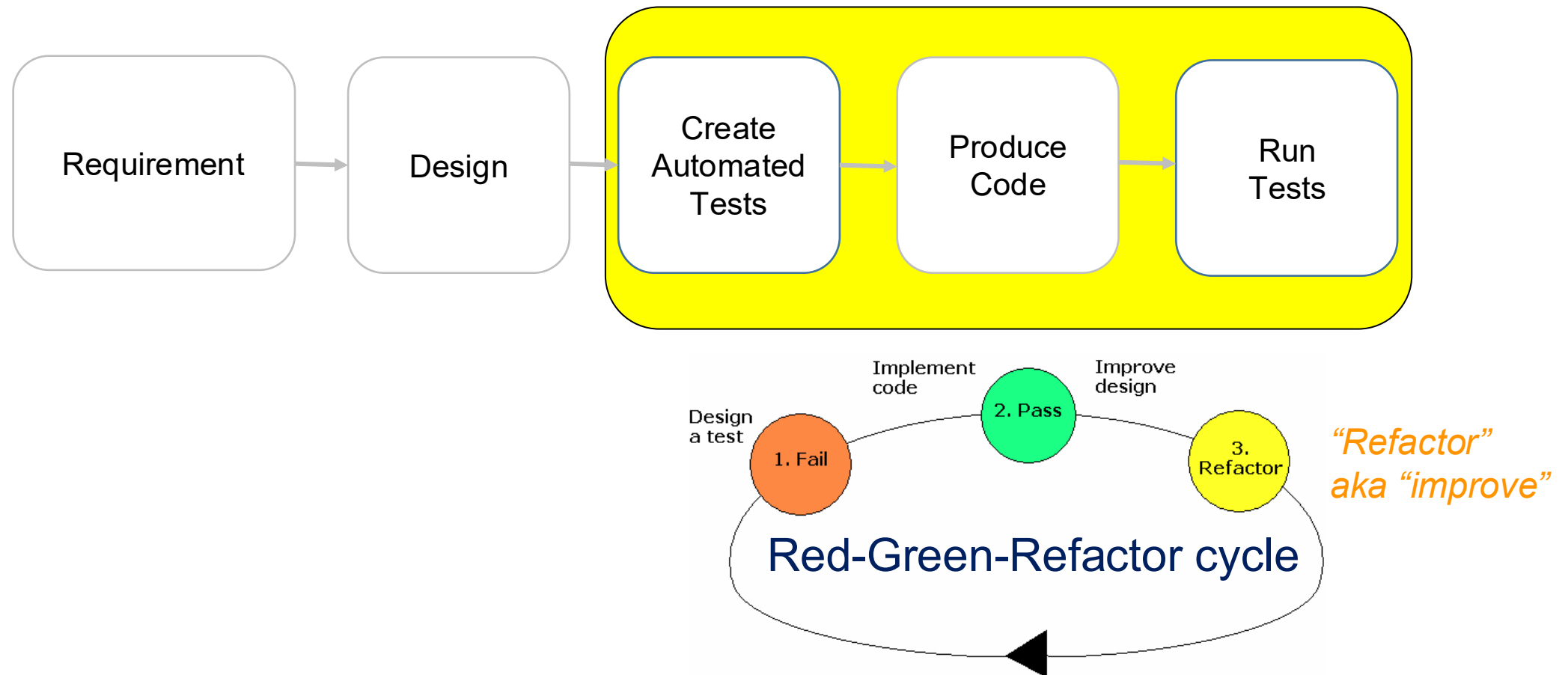
Test-Driven Development is an approach in which we **create a failing test**, then produce **just enough code to pass it**  and finally refactor while **tests stay green** 

**Specify
behaviour first**

**Automated
feedback**

The goal of TDD is 'clean code that works'

Test-Driven Development - Process



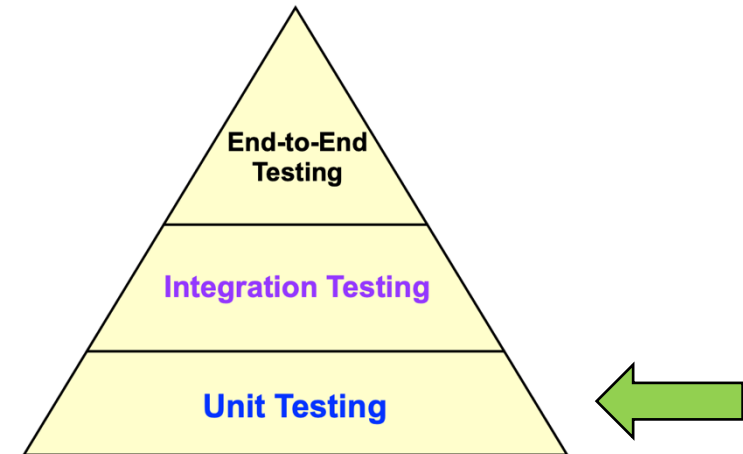
Red-Green-Refactor with an Agent

Stage	An agent can help to	Your responsibility is to
Red	propose examples and test code	confirm the required behaviour and the oracle
Green	propose a minimal implementation	reject scope creep and misunderstanding
Refactor	spot duplication, suggest restructuring	preserve intent, judge design quality

The cycle does not change. **Who decides what “correct” means does not change either.**

Unit Tests by Example

– *using TDD and `unittest`* –



Typical process of using unittest for testing

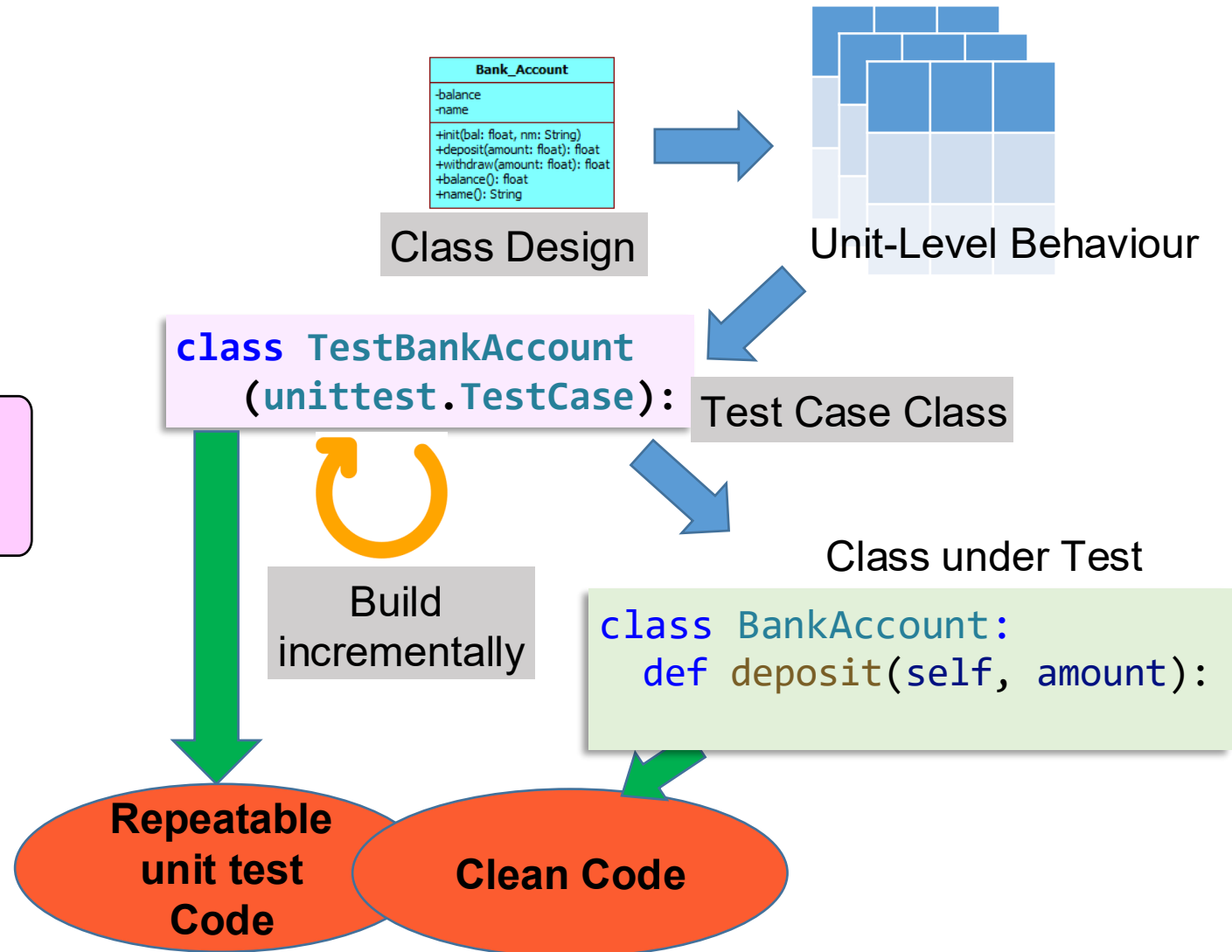
Identify **unit-level behaviours** to test

Determine the **inputs, expected outputs, edge cases**, and **error conditions** for the class/method

Build TestClass **classes** that contain **test methods** that implement these checks

In the test methods introduce **assertions** that compare the expected and actual results

Build the class-under-test to **'pass'** the test cases



TDD Example: BankAccount Class Development

Designing Unit Tests

Deposit Method:

1. Deposit amount = 0
 - Balance same as before
2. Deposit amount = positive number
 - Balance increased by the positive number
3. Deposit amount = negative number
 - Exception raised

Withdraw Method:

1. Withdraw amount = 0
 - Balance same as before
2. Withdraw amount = positive number less than balance
 - Balance decreased by the positive number
3. Withdraw amount = positive number greater than balance
 - Exception raised
4. Withdraw amount = negative number
 - Exception raised

Other test cases:

1. Test for same account objects
2. Test for different account objects
3. Test for different account objects with same attribute values

**Add test cases
iteratively**

Live Demo

*Let's produce **BankAccount** together using TDD*

BankAccount
balance: float name: str
<code>__init__(balance: float, name: str)</code> <code>deposit(amount: float)</code> <code>withdraw(amount: float)</code>

unittest framework

(the assertions you are most likely to use)

Method	What it checks
assertEqual(a, b)	<code>a == b</code> – the one you'll use most
assertTrue(x), assertFalse(x)	truthy / falsy (<i>use more specific assertion methods where possible</i>)
assertIsNone(x), assertIs(a, b)	identity, not equality
assertIn(a, b)	membership: <code>a in b</code>
assertAlmostEqual(a, b)	floats – never <code>assertEqual</code> on floats
assertRaises(Error, fn, *args)	or: with <code>self.assertRaises(Error):</code>
assertListEqual, assertDictEqual	collections, with readable failures

<https://docs.python.org/3/library/unittest.html#assert-methods>

unittest framework – Test Fixtures

- Test methods often use same test data that need to be
 - Initialised at the start of the test method
 - Uninitialised at the end of the test method
- Common **Test Data** is called **Test Fixtures**
- **unittest** provides the following two methods to initialise and uninitialised Test Fixtures
 - **setUp()** – to initialise [called before every test_method()]
 - **tearDown()** – to uninitialise [called after every test_method()]

TDD Example: **BankAccount** Class Development

Possible outcome after the Red-Green-Refactor cycle

```
class BankAccount:
    def __init__(self, bal=0, nm=''):
        self.balance = bal
        self.name = nm

    def _validate(self, amount):          # extracted during Refactor
        if amount < 0:
            raise ValueError("Amount must not be negative")

    def deposit(self, amount):
        self._validate(amount)
        self.balance += amount

    def withdraw(self, amount):
        self._validate(amount)
        if amount > self.balance:
            raise ValueError("Insufficient balance")
        self.balance -= amount
```

Clean Code



TDD Example: **BankAccount** unittest

Possible tests after the Red–Green–Refactor cycle

tests/test_bank_account.py

```
import unittest
from bank_account import BankAccount

class TestBankAccount(unittest.TestCase):
    def setUp(self):
        self.account = BankAccount(100, "Alice")

    def test_deposit(self):
        self.account.deposit(50)
        self.assertEqual(self.account.balance, 150)

    def test_withdraw(self):
        self.account.withdraw(40)
        self.assertEqual(self.account.balance, 60)

    def test_negative_amount_rejected(self):
        with self.assertRaises(ValueError):
            self.account.deposit(-10)

    def test_overdraw_rejected(self):
        with self.assertRaises(ValueError):
            self.account.withdraw(101)

if __name__ == "__main__":
    unittest.main()
```

TEST FIXTURE

setUp() creates the same starting object before every test.

What this suite checks

- ✓ Deposit updates balance
- ✓ Withdrawal updates balance
- ✓ Negative amounts are rejected
- ✓ Overdrawing is rejected

...does it cover all cases?

Each test gets a fresh BankAccount – tests stay independent

Test Coverage

Test Coverage

- **Test coverage** measures **how much of the software's code** has been “exercised” by your test suite
- There are different types of coverage:
 - **Statement coverage** – % of **executable statements** that have been executed
 - **Branch coverage** – % of **decision branches (if-then-else)** that have been executed
 - **Function coverage** – % of **functions/methods** that have been called
- Coverage is a **diagnostic, not a target**
 - Low coverage reliably tells you something has not been tested at all
 - High coverage does **not** tell you the tests are any good ([Inozemtseva & Holmes, ICSE 2014](#))
- Ask an agent to raise coverage and it will raise coverage – it can reach 100% potentially **without asserting anything useful**

Test Coverage – Exercise

```
def calculate_discount(price, discount_rate):
    """Calculate the discounted price."""
    if price <= 0:
        raise ValueError("Price must be positive.")
    if discount_rate < 0 or discount_rate > 1:
        raise ValueError("Discount rate must be between 0 and 1.")

    discount = price * discount_rate
    discounted_price = price - discount
    return discounted_price

def apply_tax(price, tax_rate):
    """Apply tax to the price."""
    if tax_rate < 0 or tax_rate > 1:
        raise ValueError("Tax rate must be between 0 and 1.")
    return price * (1 + tax_rate)
```

```
def test_calculate_discount(self):
    self.assertEqual(calculate_discount(100, 0.2), 80)
    self.assertRaises(ValueError, calculate_discount, -10, 0.2)
    self.assertRaises(ValueError, calculate_discount, 100, 1.5)

def test_apply_tax(self):
    self.assertEqual(apply_tax(100, 0.1), 110)
    self.assertRaises(ValueError, apply_tax, 100, -0.1)
    self.assertRaises(ValueError, apply_tax, 100, 1.5)
```

*What is the **statement**, **branch**, and **function** coverage?*

Test Coverage – Exercise

```
def is_palindrome(s):  
    cleaned_string = ''.join(char.lower() for char in s if char.isalnum())  
    return cleaned_string == cleaned_string[::-1]
```

*Which tests are sufficient for **100%** statement coverage?*

```
assert is_palindrome("radar")      assert not is_palindrome("Phris")  
  
assert is_palindrome("")           assert not is_palindrome("123")  
  
assert is_palindrome("A Santa lived as a devil at NASA")
```

Is 100% Test Coverage Always Possible?

Not necessarily. 100% is only possible if every measured statement / branch is reachable.

1 Unreachable code?

Some branches are logically impossible to execute:

```
def classify(x):  
    if x < 0 and x > 10:  
        return "special"  
    return "normal"
```

No value of x can make that condition true

2 What's measured?

Coverage is normally scoped to code you own:

```
import requests  
r = requests.get(url)  
return r.json()["name"]
```

- Your test executes **library code** too.
- But the target is usually your_app/
...not requests, Python, the OS, etc.

100% coverage is not always achievable – coverage depends on what code is included in the measurement

Test Coverage

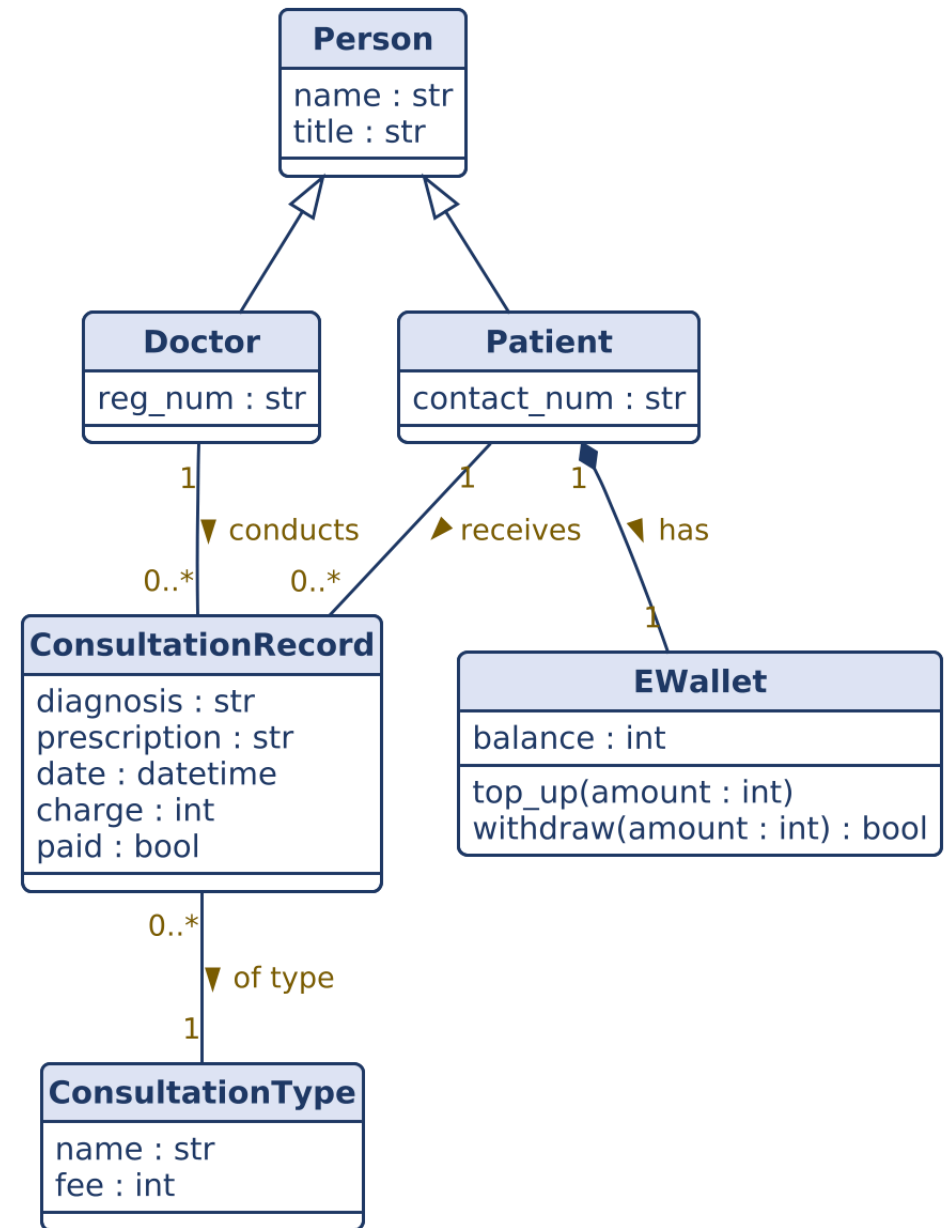
- The **Coverage.py** tool can be used to **measure the test coverage** of your Python programs
 - <https://coverage.readthedocs.io/>
- It can be executed in a terminal:
 1. **Install:** `python -m pip install coverage`
 2. **Run:** `coverage run -m unittest your_test_file.py`
 3. **Generate Report:** `coverage report`
 4. **Generate HTML Report:** `coverage html`
- **Tip:** use “`# pragma: no cover`” to exclude code from the analysis
 - <https://coverage.readthedocs.io/en/latest/excluding.html>

Demo!

Recall our Flask app

Demo!

Let's review the unit tests we generated last week...



Suppose We Asked an Agent for EWallet.withdraw()

- **Requirement:** A withdrawal must return **False** and leave the balance unchanged if the amount is **not positive** or exceeds the available balance; otherwise, deduct the amount and return **True**

```
# what the agent wrote
class EWallet:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            return False
        self.balance -= amount
        return True
```

```
# ...and the tests it wrote for it
def test_withdraw_ok(self):
    wallet = EWallet(balance=100)
    result = wallet.withdraw(30)
    self.assertTrue(result)
    self.assertTrue(wallet.balance >= 0)

def test_withdraw_too_much(self):
    wallet = EWallet(balance=100)
    result = wallet.withdraw(200)
    self.assertFalse(result)

def test_balance_after(self):
    wallet = EWallet(balance=100)
    wallet.withdraw(30)
    self.assertEqual(wallet.balance, 100 - 30)
```

100% coverage; but is the test suite good enough?

Three Mutants – Which Would Survive?

- **Mutate one thing** in the implementation. Does the test suite go **red**?
 - ***amt** > **balance** becomes **amt** >= **balance** → **SURVIVES** – withdrawing exactly the balance is never tested ☹️*
 - ***balance** -= **amt** becomes **balance** += **amt** → caught by **test_balance_after***
 - ***return False** becomes **return True** → caught by **test_withdraw_too_much***
- This is **mutation testing** – systematically injecting “bugs” into the implementation to establish whether the **test suite is able to catch them**
- Even with this, a serious logic bug still remains: **withdraw(-50) succeeds and increases the balance**

Additional Tests to Address the Gap

- **Add tests** for the **missing requirement** and the **untested boundary**
- State expected outcomes directly
- Coverage did not change (still 100%), but fault detection improved
- A strong test suite **rejects incorrect implementations**, rather than merely accepting the current one

```
def test_rejects_negative_amount(self):  
    wallet = EWallet(balance=100)  
    result = wallet.withdraw(-50)  
    self.assertFalse(result)  
    self.assertEqual(wallet.balance, 100)  
  
def test_exact_balance_is_allowed(self):  
    wallet = EWallet(balance=100)  
    result = wallet.withdraw(100)  
    self.assertTrue(result)  
    self.assertEqual(wallet.balance, 0)
```

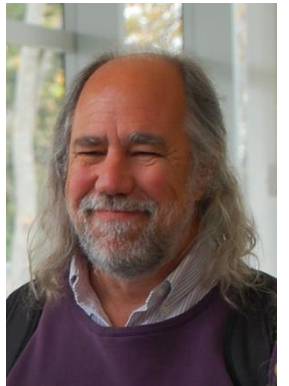
Reviewing Generated Tests – Five Questions

- **Which requirement does this test cover?** Every test **should trace back** to an acceptance criterion
- **What wrong implementation would make it fail?** If nothing plausible would, the test is **decoration**
- **Can you explain the setup, the action and the expected result?** If not, you **cannot stand behind it** in review
- **Can the expected value be justified independently of the code?** The user stories / ACs are **normally the primary source**
- **Is it deterministic and non-vacuous?** An agent reporting “all tests pass” is not evidence

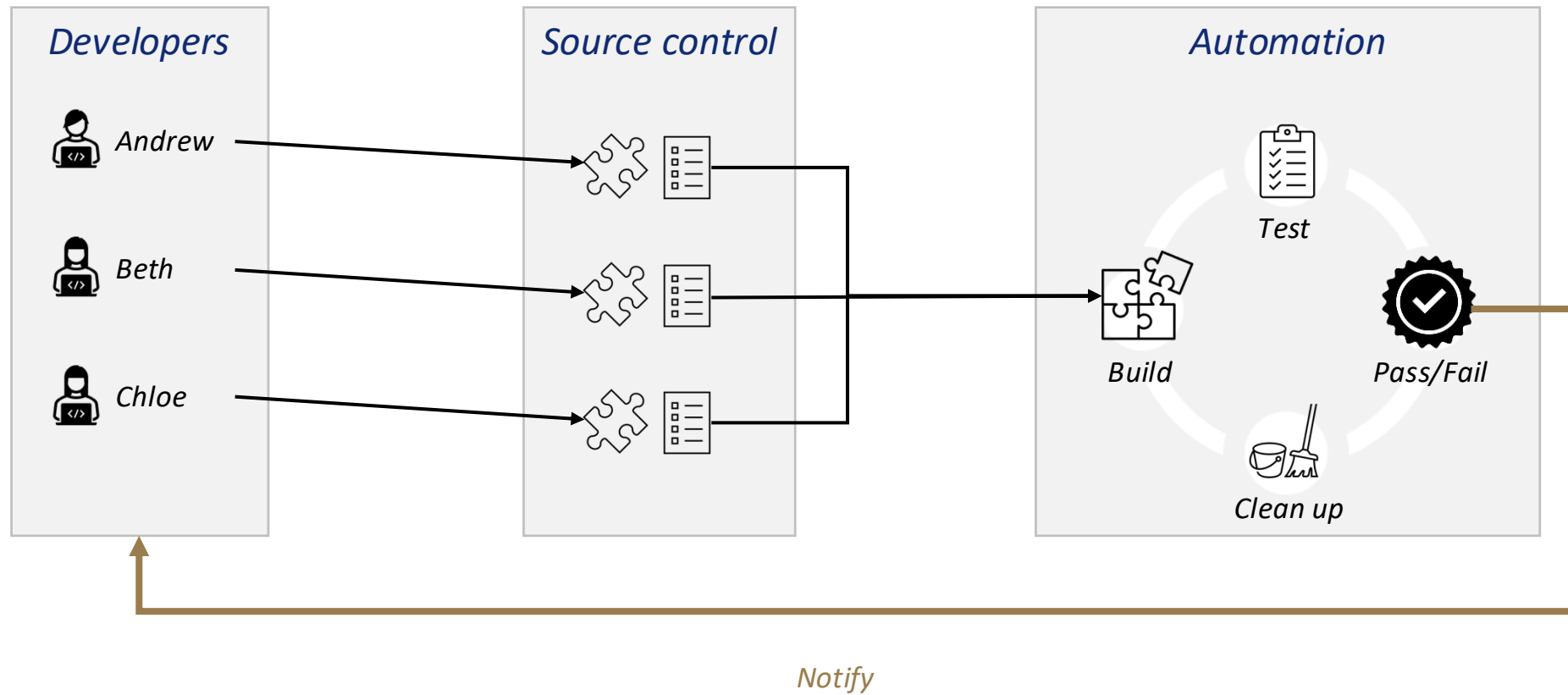
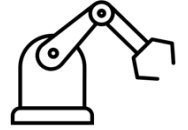
Continuous Integration (CI)

Continuous Integration (CI)

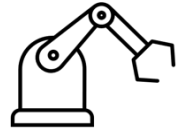
- **Continuous Integration (CI)** is the practice of **frequently merging** all developers' changes into the trunk
- Grady Booch proposed the term in 1991; it was later adopted by the XP community who advocated **integrating several times a day**
- The practice is compatible with **trunk-based development**
 - *Developers either commit small batches of changes directly to main*
 - **OR** *they work in short-lived branches (merged within hours, or < 1 day)*
- **Automated tools** are used to ascertain the new code's correctness



Automation in CI



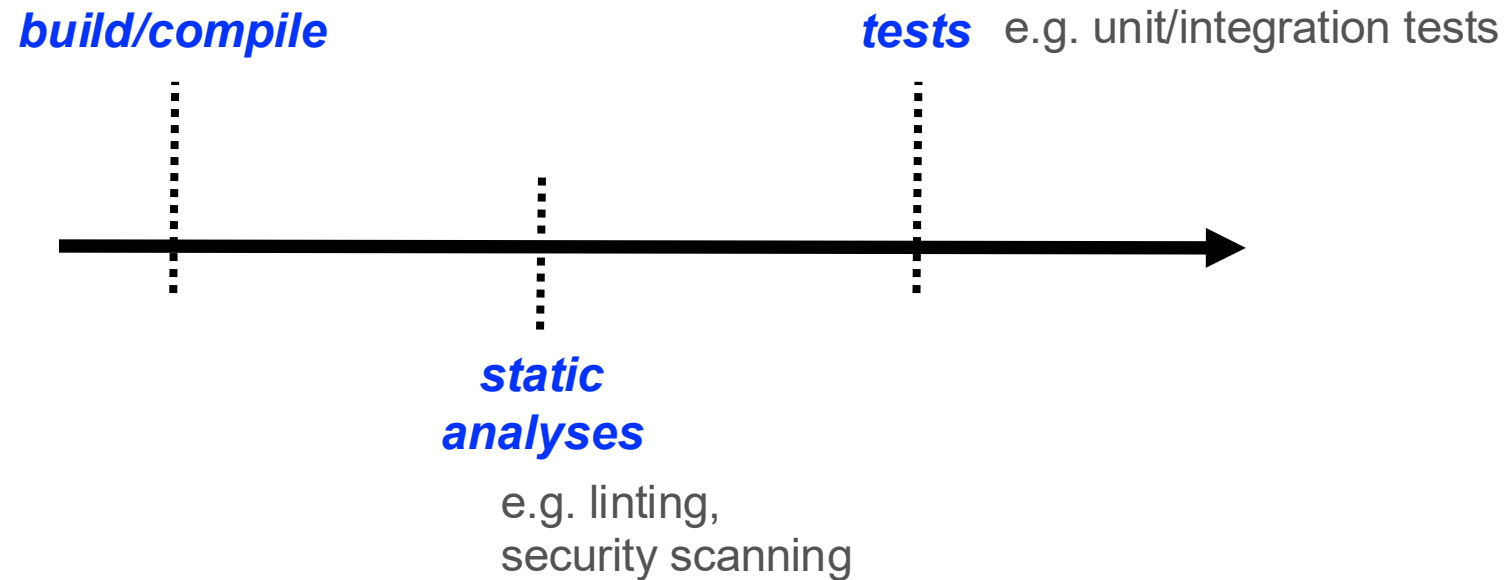
Why is Automation Valuable in CI?



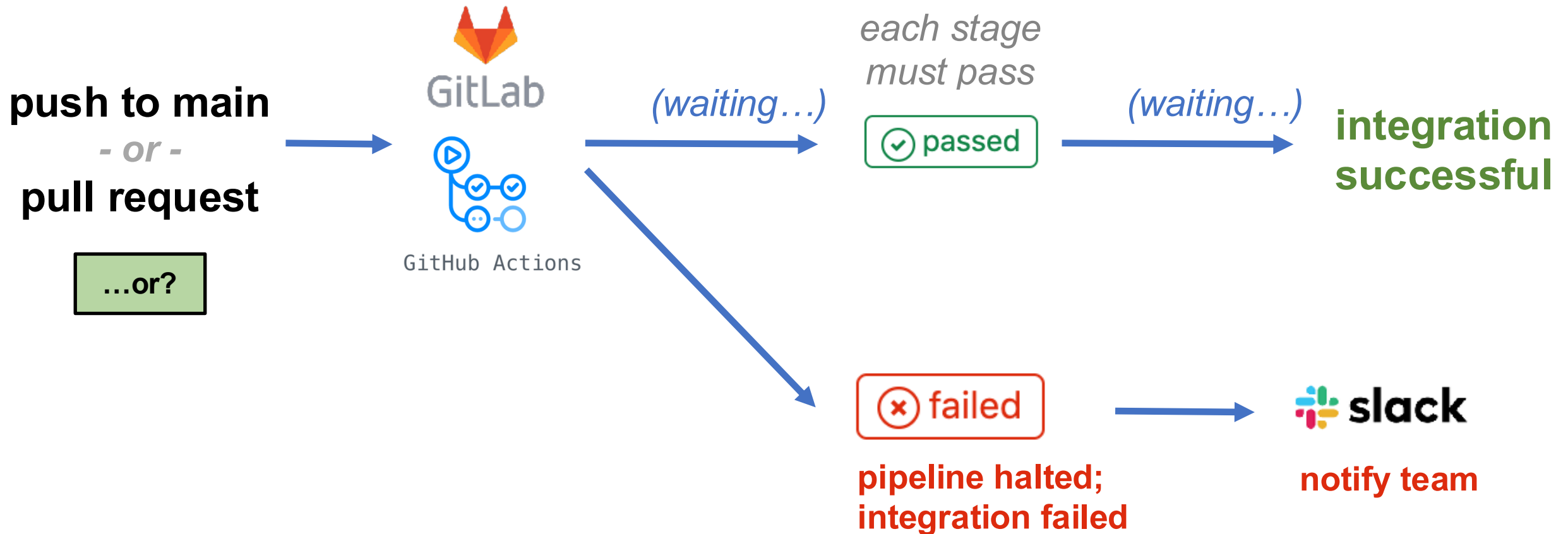
- Acts as a **consistent team member** that always runs all the required steps
- Integration explicitly follows a pre-programmed workflow (**cannot skip steps!**)
- Gives **immediate feedback** when things go wrong (e.g. changes fail tests)
- Gives developers **more confidence** in the stability of their work
- A “**safe guard**” in case developers (or agents!) forget to run tests locally
- Captures “know-how” from team members in **persisted CI pipeline scripts**

CI Pipelines

- A **CI pipeline** is an automated workflow for integrating code
- Typically consists of **several jobs/checks** assessing different aspects of code quality
- Each job runs **one or more scripts** (e.g. “python unit_tests.py”)



Triggering a CI Pipeline



When to Trigger a CI Pipeline?

- Run your tests locally first – it is the **fastest place to fix things**
- If **working with short-lived branches**, automatically trigger the CI pipeline whenever a developer creates a pull request
 - *i.e. check that all the tests pass before merging the changes*
- If **working directly on main**, automatically trigger the CI pipeline whenever a developer pushes changes to it
 - *If tests **fail**, automatically notify the team that main has become broken*
 - *Prioritise fixing or reverting the change (otherwise new work is **blocked**)*

Three Layers of Feedback

- The same tests run in more than one place, for different reasons:

1 Your machine, and your agent's loop

Fastest, but partial – and run by whoever wrote the code

2 CI on the pull request

The authoritative gate: every required check, in a clean environment, independent of the author's machine

3 Scheduled and post-merge runs

Slower, broader checks – we return to these in Week 9

Run tests locally because it is **faster** to fix things there (not because CI is expensive). An agent reporting “all tests pass” is layer 1.

CI Is the Gate on the Pull Request

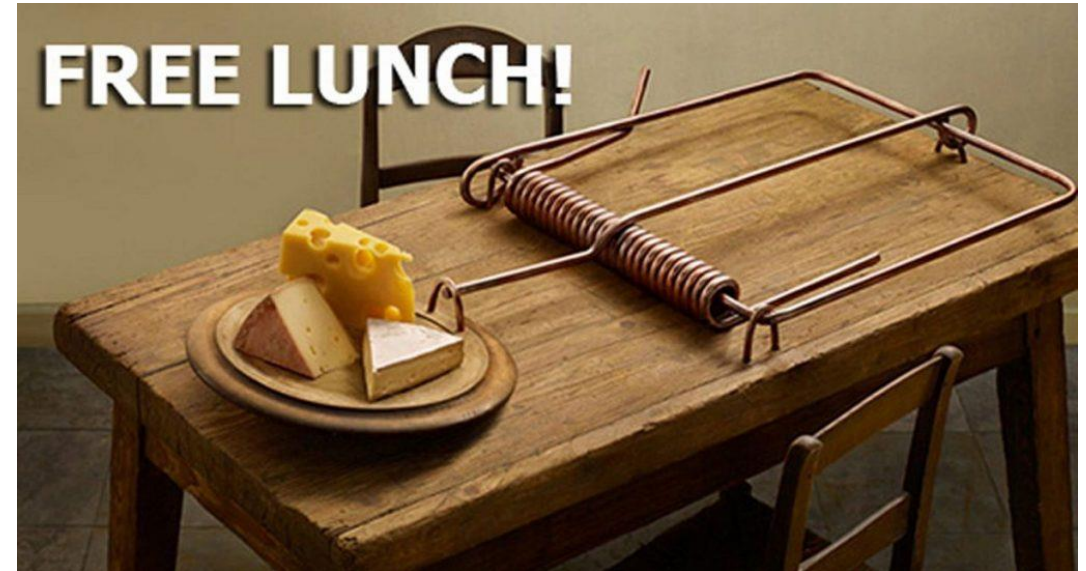
- You know how to [open a pull request](#) (Week 5) – CI turns it into a **gate** 🚧
 - *Each workflow run reports a **status check** against the pull request*
 - *Mark it **required** and the merge button stays disabled until it passes*
- This makes “[all the tests pass](#)” a **fact about the repository**, not a claim in the pull request description
- It applies to [every contributor equally](#) – a **teammate**, or an **agent** opening a pull request on your behalf
 - *Whether the change ought to be merged at all is up to the review*

Branch Protection in Practice

- On GitHub this is a **branch protection rule** (or ruleset) on **main**
- A reasonable configuration for your team project:
 - *Require a pull request before merging*
 - *Require status checks to pass – then select your CI workflow*
 - *Require the branch to be up to date before merging*
 - *Block force pushes to main*
- Without this, CI is **advisory**. With it, CI is a **gate** 🚧

There's no such thing as a free lunch

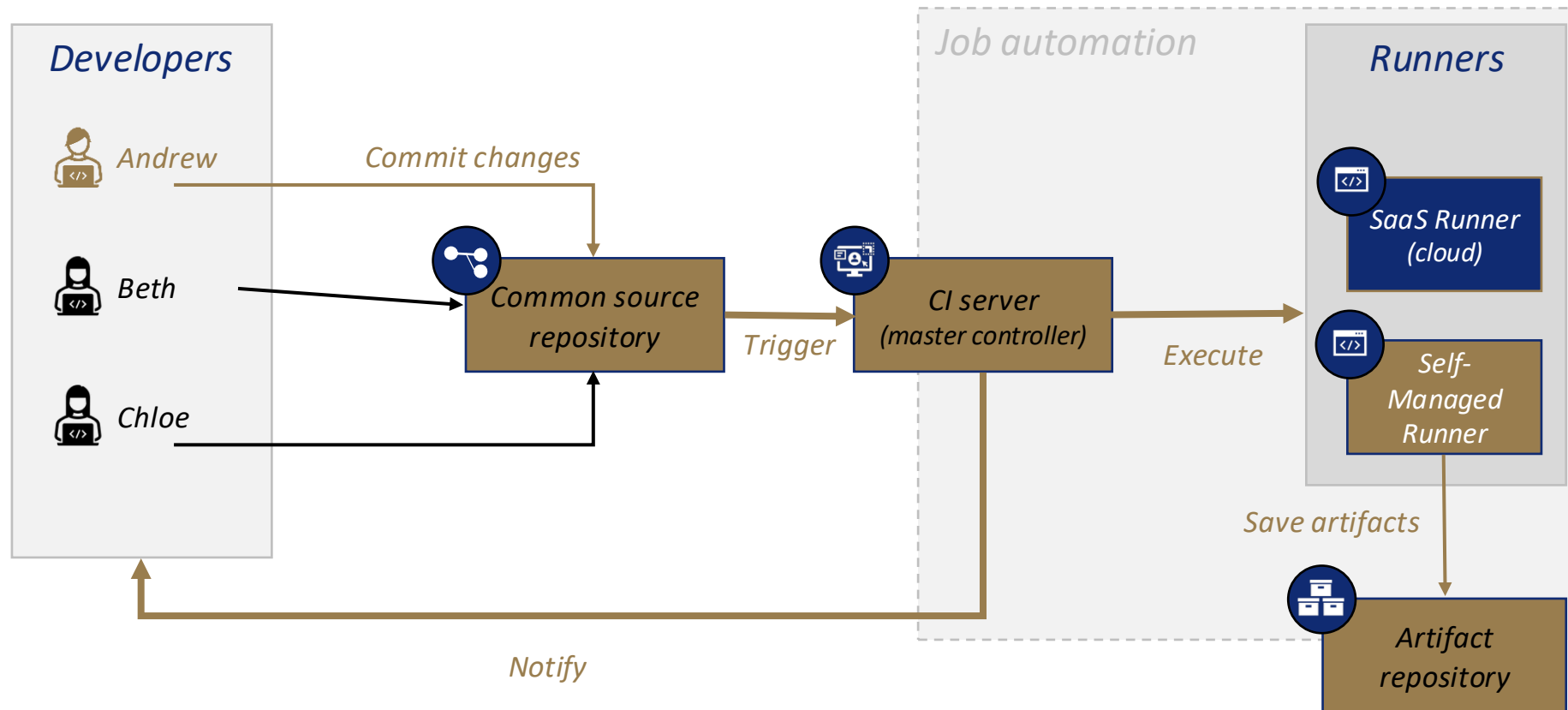
- The full benefits of CI automation **don't** come for free (sadly)
- A **strong testing culture** in the team is key to making it work
- The creation and maintenance of **automated tests** must be treated as a **core part** of your software development process
- Use coverage to **find untested code** – not as proof that the code works
 - A coverage target that is easy to hit is easy to game – by people, **and by agents**



CI Technology

CI Servers

- In general, a **CI server** consists of the following parts:



CI Platforms Differ – the Concepts Transfer

- Products come and go, but every CI platform **uses the same vocabulary**:
 - **Event / trigger** – what starts a run (push, pull request, schedule)
 - **Workflow** – the pipeline definition, versioned alongside your code
 - **Job** – one unit of work; jobs may run in parallel or depend on other jobs
 - **Runner** – the machine that executes a job (cloud or self-managed)
 - **Clean environment** – every run starts from a known state
 - **Secret** – credentials injected at run time, never committed
 - **Artifact** – saved output: logs, test reports, build outputs
 - **Status check** – the pass/fail signal a pull request can require
- Learn these once and Jenkins, GitLab CI or Azure Pipelines are easy to pick up

Our Worked Example – GitHub Actions

- **GitHub Actions** is integrated with the repository, so no separate CI server is required
- A **workflow** is a **YAML file** in `.github/workflows/` – it lives with your code, and is reviewed like code
- Jobs run on **GitHub-hosted runners** in a clean environment, on the events your workflow declares (pushes, pull requests, schedules, ...)
- GitHub plans include free Actions usage; limits and billing depend on repository visibility and plan
- Each workflow reports a **status check** – which a pull request can be **required to pass**

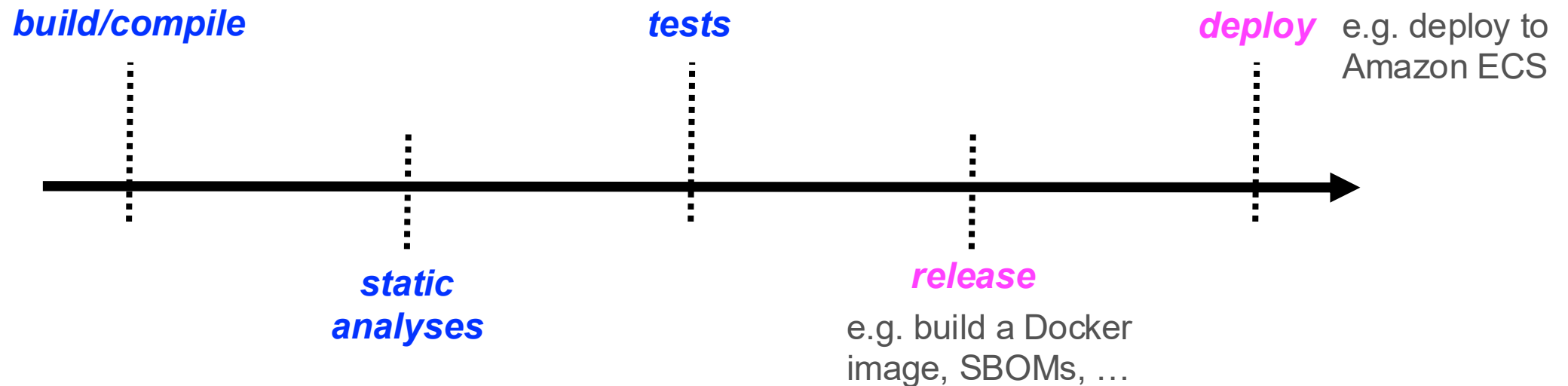
Demo!



GitHub Actions

Beyond CI – a Look Ahead to Week 9

- **Continuous Delivery / Deployment (CD)** extends the pipeline: once every check passes, the change is packaged and released automatically
- In Week 9 we will cover release pipelines, gating releases on tests, and deploying to staging and production



Lab & Assignment

Lab Exercise – CI with GitHub Actions

- Access the **Week 6 Lab Exercise** in eLearn
- It will show you how to **automate unit tests** in a **GitHub Actions** CI pipeline
- **This is for illustration only** – you can use alternative platforms (e.g. GitLab) in the course project

School of Computing and Information Systems

IS212 Software Project Management



Week 6 Lab – From Generated Tests to a CI Gate

This week we will let an agent write a test suite and then decide whether to believe it. Generating tests is close to free now; knowing whether they would catch a fault is not.

In the second half we push the code to GitHub, turn that suite into a CI workflow, and make it a gate, i.e. a pull request that breaks a test cannot be merged.

Week 5 got your code into a repository behind a protected main branch. Today we add the piece that makes the protection mean something: a required status check.

Before You Start

1. An **agentic coding tool** of your choice (GitHub Copilot, Windsurf, Cursor, Claude Code, Gemini CLI, ...). If yours misbehaves today, generate the tests in a plain chat assistant (e.g. SMU's Gemini subscription) and paste them in: the point is the workflow, not the tool.
2. Python 3 and Git installed locally (`git --version` to check), and a GitHub account.
3. Nothing to install beyond that, and **nothing to submit**.

Individual Assignment #2 – Automated Testing

- The second **individual assignment** is an **automated testing exercise** – to be released on Thursday Week 6, 18:45
- Your task is to **write/generate/review unit tests for some provided code** that achieve high code coverage and can detect injected faults
- Worth **2.5%** of the grade for IS212
- Assignment is **auto-graded**, so please follow the instructions carefully
 - Your tests **must** all run and pass on our code (as instructed), otherwise they **receive 0 marks**

IS212 (AY 2026-27, T1) – Week 6: Automated Testing Assignment

Background

You have been hired as a Software Engineer in Test by **Air Britannia**, a long-established airline flying from London to destinations across Europe, Asia and the Americas.



Air Britannia has developed a Python application that calculates the miles earned by members of its **Britannia Club** loyalty programme. Every sector flown earns two separate figures: **redeemable miles**, which members spend on reward flights, and **elite miles**, which count towards the member's tier for the following year. The two figures are not the same, and some sectors earn nothing at all.

Due 23:59 on Friday Week 7
(1 sec late = 50% pen.)

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet 🖥️
- <http://smu.sg/is212-reflections-2026>

